SCALECUA: Scaling Computer Use Agents with Verifiable Task Synthesis and Efficient Online RL

Bowen Lv, Xiao Liu, Yanyu Ren, Hanyu Lai, Bohao Jing, Hanchen Zhang, Yanxiao Zhao, Shuntian Yao, Jie Tang, Yuxiao Dong

Paper version: arXiv 2607.11185v1 · CUA Radar Reel poster, not made or endorsed by the authors

Problem

> *Scaling online RLVR for CUAs remains largely unsolved due to verifiable data scarcity and online RL inefficiency.*

A GUI task has a live environment and an instruction, but **no built-in judge**; the reward must be built, at the right difficulty.

Motivation: three open gaps

1. **No scorer.** Tasks need a deterministic judge.
2. **Wasted rollouts.** All-pass or all-fail: zero advantage.
3. **Slow training.** Step-wise items grow with turns.

Evaluation setup

OSWorld · online | ScienceBoard · online | Qwen3.5-9B base | GLM-4.6V-Flash | Qwen3-VL-8B | 50-turn cap | 4 rollout passes | 64 H800 GPUs

- **All online**; no offline accuracy.
- **Not held out:** tasks come from the same OSWorld / ScienceBoard apps.

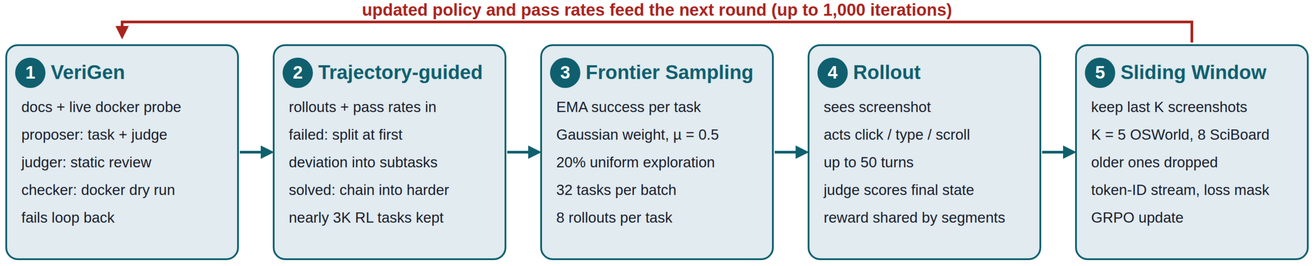
Our read (CUA Radar)

76	★★★	online
score of 100	band	evidence level

- **Reports:** online eval; 4-pass std; sampler baselines; cost; ablation; judge audit; code.
- **Flags:** tasks span all 10 OSWorld and 6 ScienceBoard domains; held-out env, budget match partial.
- Pattern K13. Authors' numbers, not rerun.

Method: synthesise judged tasks, then train online with a sliding visual window

- **Sees / acts:** screenshots only; mouse, keyboard and scroll at (x, y) on a 1000×1000 grid.
- **Platform:** Ubuntu desktop VMs in docker (OSWorld, ScienceBoard).
- **Signal:** a Python judge scores the final state; GRPO with DAPO-style clipping.

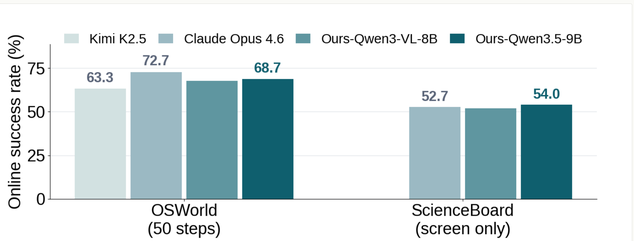


Our redraw (not the paper's figure): synthesis, sampling and training loop as described in the method text and appendix.

$$w_i = \exp\left(-\frac{(\hat{p}_i - \mu)^2}{2\sigma^2}\right)$$

Frontier weight: peaks at target success $\mu = 0.5$, $\sigma = 0.25$

Key results: online success rate (%)



Tables 2 and 3, redrawn. Ours: mean of 4 rollout passes (std 0.5 to 1.5).

68.7 vs Kimi K2.5 63.3 (OSWorld); 54.0 vs Opus 4.6 52.7 (SciBoard)

SO WHAT → *Kimi K2.5 is the best open-source baseline; Opus 4.6 is proprietary and scores 72.7 on OSWorld.*

Ablation: OSWorld success rate (%)

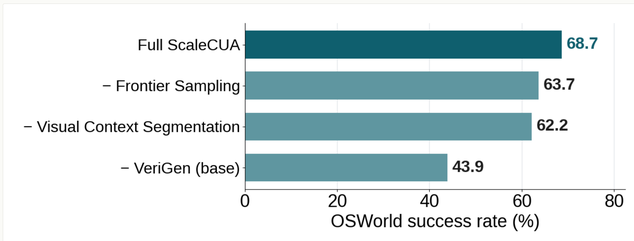


Table 4, redrawn: Qwen3.5-9B backbone, online OSWorld, one component removed at a time. The VeriGen row is labelled "Base" in the paper.

SO WHAT → *Removing VeriGen returns the 43.9 base score; dropping Frontier Sampling (63.7) or Visual Context Segmentation (62.2) alone leaves the model well above it. Each row is one reported score, with no std; the sampler comparison itself is shown only as curves. Table 2 lists 41.8 for the Qwen3.5-9B base (technical report).*

Headline Numbers

68.7%
OSWorld success rate
Qwen3.5-9B; base 41.8%; 50 steps

- **54.0%** ScienceBoard
- **24K+** verifiable tasks
- **2.83×** training speedup

The paper's own controls

CONTROL	SCALECUA	COMPARED WITH
Judge vs human (160)	82.5% agree	OSW 78.0 / SB 90.0
Trajectory-guided tasks	68.7 OSWorld	64.6 without them
Eval-overlap audit	0.19% OSW near-dup	SB 0.36%
Step time, same batch	265 s (K = 5)	750 s step-wise

Takeaway

Executable judges, frontier sampling and a sliding window lift Qwen3.5-9B from 41.8 to 68.7 on OSWorld; tasks come from the same apps.

24K+ candidate tasks | nearly 3K RL tasks | 8 nodes × 8 H800

Scan to Read

Paper | Code